4. 상태 관리 및 저장 프로세스

개요

Sapphire ParaTime에서 스마트 컨트랙트 상태가 어떻게 암호화되어 관리되고 저장되는지,

그리고 기밀성을 유지하면서도 효율적인 상태 접근을 제공하는 메커니즘을 설명합니다.

4.1 상태 관리 아키텍처

계층적 상태 구조

```
┌─────────────────────────────────────────────────────────────┐
│                     Application Layer                       │
│ ┌─────────────────────────────────────────────────────────┐ │
│ │                     Smart Contracts                     │ │
│ │ ┌─────────────────────────────────────────────────────┐ │ │
│ │ │                    Contract State                   │ │ │
│ │ │             - Storage Slots (encrypted)             │ │ │
│ │ │             - Balance (encrypted)                   │ │ │
│ │ │             - Code Hash                             │ │ │
│ │ └─────────────────────────────────────────────────────┘ │ │
│ └─────────────────────────────────────────────────────────┘ │
│ ┌─────────────────────────────────────────────────────────┐ │
```

```
│ │                    Account State                    │ │
│ │                - Nonce                              │ │
│ │                - Balance (encrypted)                │ │
│ │                - Storage Root (encrypted)           │ │
│ │                - Code Hash                          │ │
│ └─────────────────────────────────────────────────────┘ │
│ ┌─────────────────────────────────────────────────────┐ │
│ │                     Global State                    │ │
│ │                - State Root Hash                    │ │
│ │                - Block Number                       │ │
│ │                - Timestamp                          │ │
│ └─────────────────────────────────────────────────────┘ │
└─────────────────────────────────────────────────────────┘
```

상태 버전 관리

```
const STATE_VERSION: u32 = state_version();
```

```
const fn state_version() -> u32 {
if is_testnet() {
11 // 테스트넷: 버전 11
} else {
7 // 메인넷: 버전 7
```

```
}

}
```

4.2 암호화 상태 저장

상태 암호화 메커니즘

1. **TEE 내부 처리**

- 평문 상태는 TEE 내부에서만 접근 가능
- 메모리 내에서 평문으로 처리
- 외부 저장 시 자동 암호화

2. **암호화 알고리즘**

- **대칭 암호화**: AES-256-GCM
- **키 유도**: HKDF-SHA256
- **무결성 보장**: HMAC-SHA256

3. **키 관리**

- **마스터 키**: TEE 내부에서 생성 및 관리
- **상태별 키**: 각 상태 항목별 고유 키 유도
- **키 회전**: 정기적인 암호화 키 갱신

저장 구조

```
// 계정 상태 구조
pub struct AccountState {
```

```
pub nonce: u64,
pub balance: EncryptedBalance, // 암호화된 잔액
pub storage_root: EncryptedHash, // 암호화된 저장소 루트
pub code_hash: Hash, // 코드 해시 (공개)
}

// 암호화된 저장소 슬롯
pub struct EncryptedStorageSlot {
pub key: Hash, // 슬롯 키 (해시)
pub value: EncryptedValue, // 암호화된 값
pub version: u64, // 버전 번호
}
```

4.3 상태 접근 패턴

읽기 작업

1. **상태 요청**

- 클라이언트가 상태 조회 요청
- TEE 노드가 요청 수신

2. **복호화 및 처리**

- TEE 내부에서 암호화된 상태 복호화
- 요청된 상태 값 추출
- 필요 시 계산 수행

3. **응답 생성**

- 결과를 적절한 형태로 암호화
- 클라이언트에게 응답 전송

쓰기 작업

1. **트랜잭션 실행**

- 스마트 컨트랙트 실행 중 상태 변경
- TEE 내부에서 평문 상태 수정

2. **변경사항 추적**

- 수정된 상태 항목 추적
- 이전 값과 새 값 비교
- 변경 로그 생성

3. **암호화 및 저장**

- 변경된 상태 암호화
- 영구 저장소에 기록
- 상태 루트 해시 업데이트

4.4 상태 동기화

노드 간 동기화

```
// 상태 마이그레이션 함수
fn migrate_state<C: sdk::Context>(_ctx: &C) {
let genesis = Self::genesis_state();
```

```
// 각 모듈별 상태 동기화

modules::core::Module::<Config>::set_params(genesis.0.parameters);

modules::accounts::Module::set_params(genesis.1.parameters);

modules::consensus::Module::set_params(genesis.2.parameters);

modules::consensus_accounts::Module::<modules::consensus::Module>::set_params(ge

modules::rewards::Module::set_params(genesis.4.parameters);

modules::rofl::Module::<Config>::set_params(genesis.5.parameters);

module_evm::Module::<Config>::set_params(genesis.6.parameters);

module_rofl_market::Module::<Config>::set_params(genesis.7.parameters);

}
```

동기화 프로세스

1. **상태 스냅샷**

- 현재 상태의 스냅샷 생성
- 암호화된 형태로 저장
- 체크섬을 통한 무결성 검증

2. **증분 동기화**

- 마지막 동기화 이후 변경사항만 전송
- 블록별 상태 변경 델타 계산
- 효율적인 대역폭 사용

3. **검증 및 적용**

- 수신된 상태 변경사항 검증

- TEE 내부에서 상태 적용
- 상태 루트 해시 확인

4.5 성능 최적화

캐싱 전략

1. **메모리 캐시**

- 자주 접근하는 상태의 메모리 캐싱
- LRU (Least Recently Used) 정책
- TEE 메모리 제한 고려

2. **디스크 캐시**

- SSD 기반 고속 캐시
- 암호화된 상태의 임시 저장
- 배치 처리를 통한 I/O 최적화

3. **프리페칭**

- 예상되는 상태 접근 패턴 분석
- 미리 필요한 상태 로드
- 지연시간 최소화

압축 및 최적화

```
// 저장소 최적화 설정
gas_costs: modules::core::GasCosts {
tx_byte: 1,
```

```
storage_byte: 15, // 저장 비용 최적화

auth_signature: 1_000,

auth_multisig_signer: 1_000,

callformat_x25519_deoxysii: 10_000,

}
```

4.6 백업 및 복구

자동 백업

1. **정기 백업**

- 설정된 간격으로 자동 백업
- 암호화된 상태 전체 백업
- 여러 위치에 분산 저장

2. **증분 백업**

- 변경된 상태만 백업
- 효율적인 저장 공간 사용
- 빠른 백업 완료

3. **백업 검증**

- 백업 데이터 무결성 검증
- 정기적인 복구 테스트
- 백업 메타데이터 관리

재해 복구

1. **장애 감지**

- 상태 손상 자동 감지
- 체크섬 불일치 모니터링
- 알림 및 로깅

2. **자동 복구**

- 백업에서 자동 복구
- 최소 다운타임 보장
- 서비스 연속성 유지

3. **수동 복구**

- 관리자 개입이 필요한 경우
- 상세한 복구 절차 제공
- 데이터 손실 최소화

4.7 모니터링 및 분석

상태 메트릭

1. **크기 메트릭**

- 총 상태 크기
- 계정별 상태 크기
- 증가율 추적

2. **접근 패턴**

- 읽기/쓰기 빈도
- 핫스팟 식별

- 성능 병목 분석

3. **암호화 오버헤드**

- 암호화/복호화 시간
- CPU 사용률
- 메모리 사용량

알림 시스템

```
// 상태 관리 파라미터
max_tx_size: 128 * 1024, // 최대 트랜잭션 크기
max_tx_signers: 3, // 최대 서명자 수
max_multisig_signers: 8, // 최대 멀티시그 서명자 수
```

4.8 보안 고려사항

접근 제어

1. **권한 기반 접근**

- 역할별 상태 접근 권한
- 최소 권한 원칙 적용
- 감사 로그 유지

2. **시간 기반 제어**

- 접근 시간 제한
- 세션 타임아웃
- 비활성 계정 관리

3. **지리적 제한**

- 지역별 접근 제한
- IP 기반 필터링
- VPN 감지 및 차단

암호화 보안

1. **키 보안**

- 하드웨어 보안 모듈 (HSM) 사용
- 키 분할 및 분산
- 정기적인 키 회전

2. **알고리즘 보안**

- 최신 암호화 표준 사용
- 양자 내성 알고리즘 준비
- 정기적인 보안 감사

결론

Sapphire의 상태 관리 및 저장 프로세스는 TEE 환경의 하드웨어 보안과

고급 암호화 기술을 결합하여 완전한 기밀성을 보장하면서도

높은 성능과 가용성을 제공합니다. 효율적인 캐싱, 압축, 백업 전략을 통해

대규모 dApp의 요구사항을 충족할 수 있는 확장 가능한 솔루션을 제공합니다.